1. Worst case and the input that triggers it

With whole-number capacities every augmenting path adds at least one unit, so Ford-Fulkerson needs at most F augmentations and runs in O(F·E). F depends on the capacity values, not just the graph's shape. The textbook trigger is a four-vertex "diamond": two routes of capacity M joined by a bridge of capacity 1. If the paths alternate through the bridge, each push moves one unit and the algorithm needs 2M augmentations, while Edmonds-Karp needs 2. I learned that this is not automatic. Plain DFS needs at most 4 augmentations on the diamond, because the first path it finds already uses a big edge. So the worst case belongs to the path choice, not to DFS itself, and I could only show it in a test where I chose the paths by hand. On realistic input the trigger was large capacities on ordinary random graphs. At C = 10,000 my Ford-Fulkerson pushed 4,442 paths with about 3 units each (13,278 units in total), Edmonds-Karp pushed 9 paths, and Ford-Fulkerson was 452 times slower

2. Hardest bug or design decision

The hardest design decision was how to show that worst case. I could have added an "adversarial" path chooser that always takes the path with the smallest bottleneck, which does produce 2M augmentations. I dropped it because I wanted code I could explain line by line. Instead, I relied on the hand-traced diamond test plus the capacity sweep, which shows the same growth on ordinary graphs. The bug that actually stopped me was in the analysis, not the algorithm. Running python3 run_all.py --quick ended with a StatisticsError ("no median for empty data"). The cost-model check fits on seeds 0 and 1 and tests on seeds 2 and up, but quick mode only has seeds 0 and 1, so the test set was empty. I fixed it by skipping that check when no test seeds exist. The lesson was to run the short mode from a clean copy before trusting a pipeline

3. What I would do differently

First, I would record the bottleneck of every pushed path. Right now I only infer that Ford-Fulkerson's pushes are small, from F divided by the number of paths. Saving the bottlenecks would let me test the reason directly. Second, I would shuffle the order of each vertex's arcs, to check that the very long DFS paths (on average 1,276 edges on a graph with 1,600 vertices) come from depth-first search itself and not from my storage order. Third, I would write my predictions before the first run. Mine were written after I had seen early results, so they are not blind

4. Something I did not fully understand before

I thought Ford-Fulkerson was one algorithm. It is a method that leaves the choice of path open, and Edmonds-Karp is one rule for making that choice. I also did not realise how much the bound matters in practice. Ford-Fulkerson's path count equals F on unit-capacity matching and on sparse graphs with 800 or more vertices (for example 783 paths for F = 783), so it runs at its worst-case bound, while Edmonds-Karp uses less than one percent of its own. The last piece was the reverse arc. A later path can push flow backwards along the paired arc e ^ 1 to undo an earlier choice, which keeps the result a maximum flow. Deleting that one line makes 8 of my 19 tests fail